CIE

COPYRIGHT INFORMATION EXTRACTION FORM

TITLE: Real-Time Collaborative Development Environment

SCHOOL: Chitkara University, Rajpura, Punjab

DETAIL OF THE STUDENT/MENTOR WHO UPLOADED THE COPYRIGHT ON GOVT PORTAL NEED TO SHARE LOGIN AND PASSWORD WITH MENTORS

Name: Durgesh Kumar

Roll No/ Employee Code: 2210991551

All MEMBERS DETAILS INCLUDING SUPERVISOR

NAME	EMP CODE/ROLL	ADDRESS WITH EMAIL AND MOBILE NO.
Durgesh Kumar	2210991551	At-Gopalpur,Post-Akopur, P.S-Cheriya Bariyarpur,Dis – Begusarai ,Bihar , Pin-848202 , mob – 8595359449 , email – durgeshchaudhary0111@gmail.com
Mentor Dr. Shikha Tuteja		Email - shikha.1290@chitkara.edu.in, Mob no - 9915369888

Write Abstract in 380-420 Words.

ABSTRACT

(Total Recommended Length: 380–420 words; Font: Times New Roman | Size: 12 pt | Spacing: Single; Follow the following Abstract Structure (Step-by-Step) for guidelines, do minor change as per need

1.1 Problem Background *(80–100 words)*

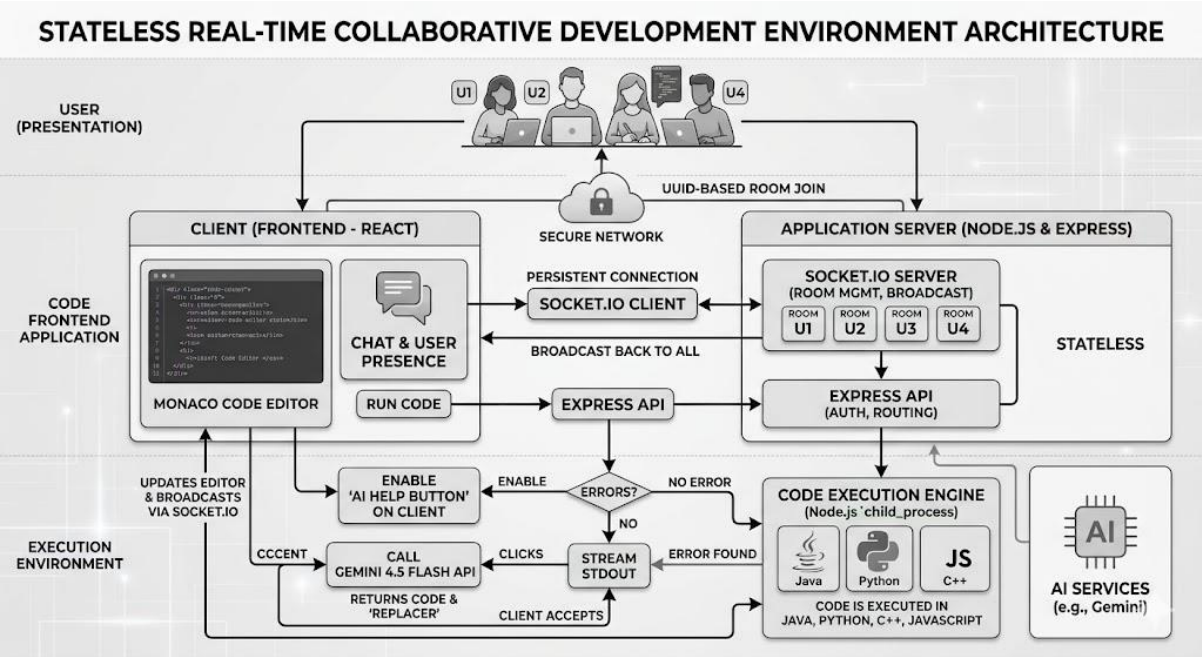
Remote collaboration while programming usually lacks the instant feedback available in the physical space, leading to disconnection while working as a group to develop software. The online code editor either does not provide any help with difficult errors or offers too much help from AI to solve the problem immediately. Over-dependence on AI hinders the students from developing logical skills and gaining mastery over "hand practice." In addition, the lack of any kind of messaging service embedded in a compiler makes users move between several platforms.

1.2 Objective of the Project *(40–50 words)*

The development of a fast-paced, MERN-stack collaborative IDE that enables real-time synchronization, a ChatBox for communication between users, and voice chat capabilities would be necessary. The system includes Grok 4.5 AI which can be unlocked only in the event of compilation failures, ensuring that students practice using their hands and think logically.

1.3 Proposed Solution / System Overview *(90–110 words)*

The system under construction is a high performing, full stack application based on the MERN architecture and employs Socket.io to create a highly efficient and bi-directional communication link between the server and multiple client machines. User identities are verified and organized into rooms based on the unique UUID identifiers in which code and messages are synced in real time. The front end includes an advanced Monaco Editor whereas the back end incorporates the Node.js child process module to execute code in Java, Python, C++, and JavaScript in a safe environment. Agora RTC technology has been embedded in the platform to enable real time audio chatting among individuals belonging to one specific room to foster effective teamwork. An interesting condition in the Conditional AI Debugger based on Grok technology locks down the debugger until an error in code compilation has occurred.



1.4 Key Features or Functionality *(60–80 words)*

Real-time Sync and Collaboration: Provides instantaneous multi-user editing using **Socket.io**, enhanced with **WebRTC-powered** high-quality audio chat and an integrated Chatbox for seamless, low-latency communication between team members.

Conditionally Locked AI Debugger: An intelligent AI assistant that unlocks exclusively during compilation failures. This design encourages independent problem-solving and "hands-on practice" by preventing over-reliance on AI for routine coding.

Smart File Handling: Enables multi-language support (Java, Python, C++, JS) with automated script execution. Users can upload existing files or export their entire collaborative project directly to their local system.

Easy Invitation via SendGrid: Streamlined project sharing through secure **SendGrid-powered email invitations**. Collaborators can join a workspace instantly via unique UUID-enabled room links with a single click.

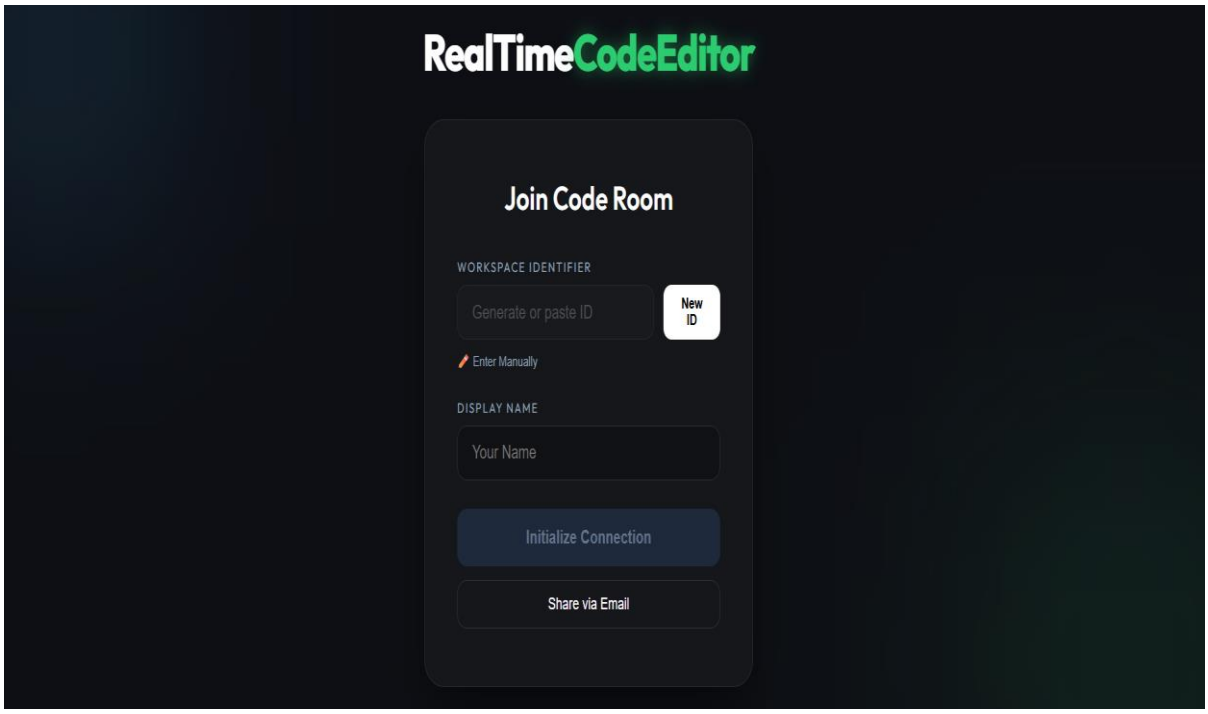


Figure 1 – This is the main point of access through which collaboration is achieved, and users have to provide two pieces of information; a unique Identifier, which is the Workspace Identifier (which can be automatically generated by clicking on the **'New ID'** button), and the Display Name which represents their username. The action button that reads 'Initialize Connection' creates a Socket.io secure connection, while another button labeled **'Share via Email**' can be used to invite more people to the programming platform.

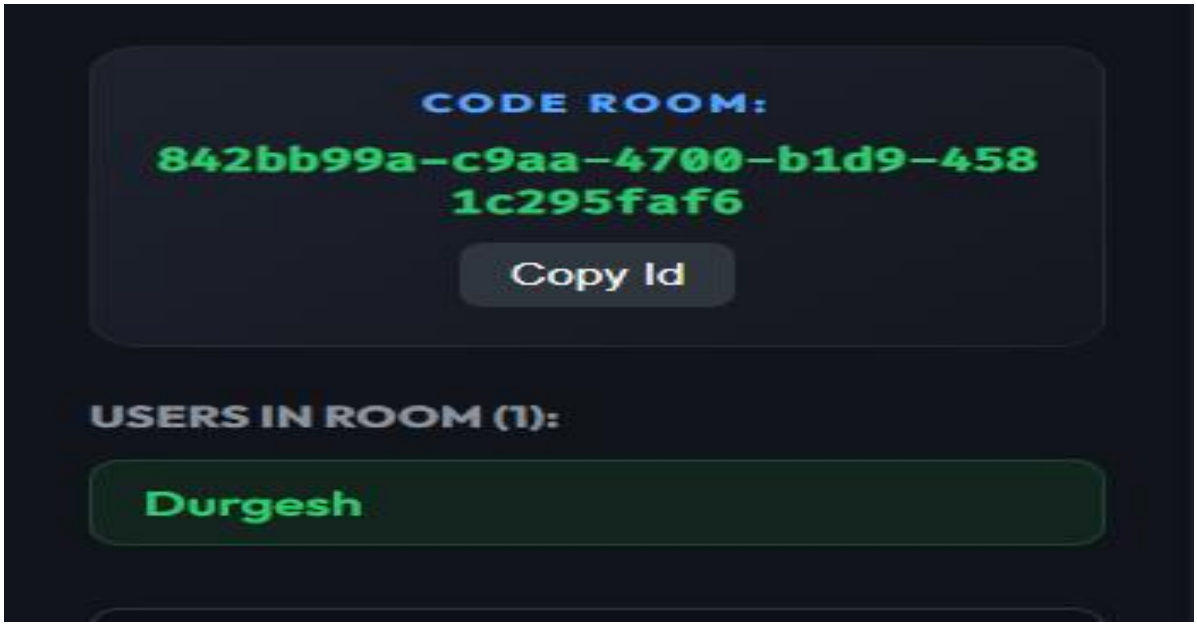


Figure 2: The **Active Room Dashboard**, showing the unique session ID and the **real-time list of users** currently collaborating in the editor.

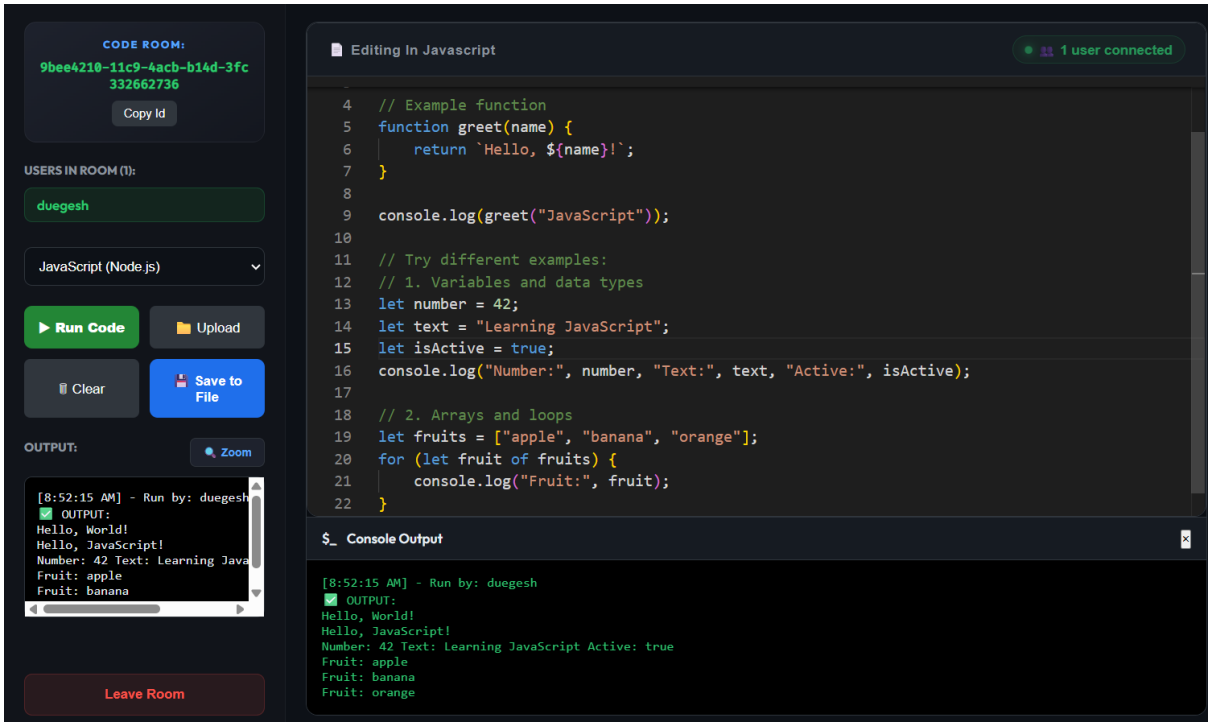


Figure 3: This perspective presents the high-end Monaco Editor with live syntax highlighting and live synchronization. On the sidebar, the controls include a **Language Selector** to support Java, Python, C++, and JavaScript, as well as buttons for uploading files in multiple languages and **saving the workspace locally**. The output console is synchronized to show live code execution upon clicking the Run Code button. The UI also includes the room management component, which shows the Room ID, a unique identifier generated using **UUID**, along with its occupants.

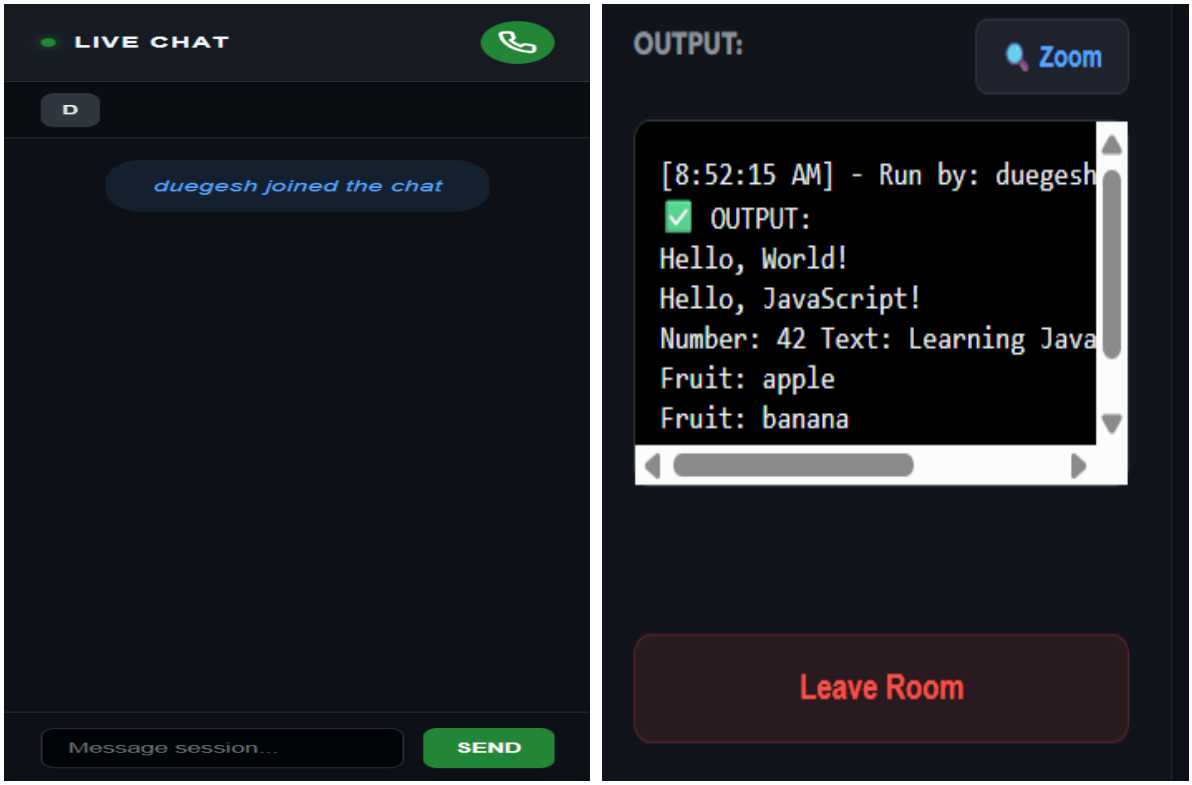


Figure 4: The Live Chat feature allows immediate interaction between peers, where the green icon for making **voice calls** via Agora is displayed. Next to it, there is the **Output Console** that shows the outcome of the code execution, along with time and the name of the executor. The purpose of this section is to ensure clear communication within the team and offer a "**Leave Room**" button.

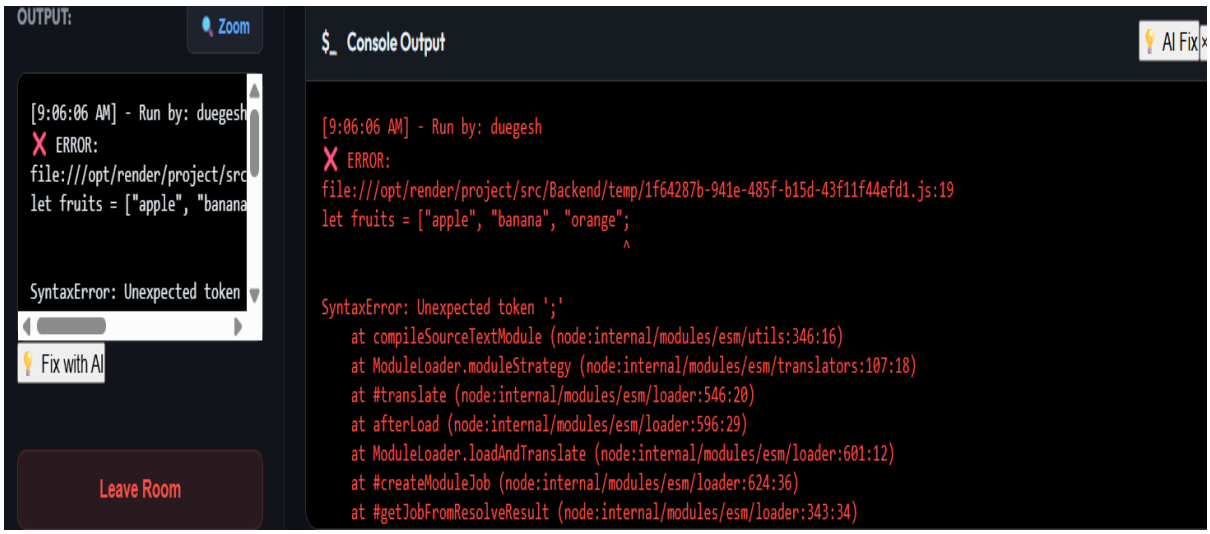


Figure 5: The diagram below shows how the conditional logic works. The "**Fix with AI**" button gets unlocked only after there is an error detected by the console, as shown above in the error message window. Limiting the use of **Grok 4.5** to troubleshooting guarantees that users focus on developing logical reasoning and problem-solving skills without the help of AI, and the latter is used solely for mentorship purposes.

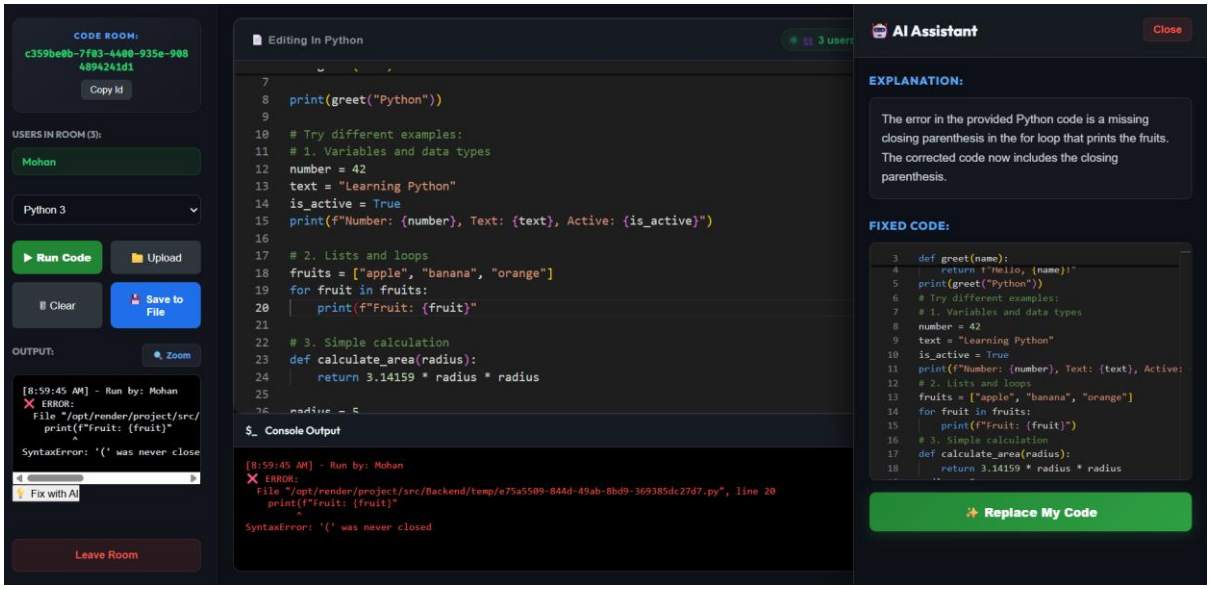


Figure 6: The interface demonstrates the system's real-time debugging capability. When a syntax error is detected in the console, the **'Fix with AI'** button becomes active. Upon clicking, an **AI Assistant panel** appears on the right, providing a detailed explanation of the bug along with the corrected code. Users can seamlessly update their project using the **'Replace My Code'** feature.

1.5 Technology / Method Used *(40–50 words)*

The application is constructed using the **MERN** technology stack, where React.js is used for creating a responsive interface, while S**ocket.io** ensures low-latency transmission of data in real-time. The Node.js/Express server-side takes care of safe code execution through the use of the **child_process** package and Ag**ora RTC** to allow smooth voice interaction between the users. **Grok 4.5** is included to offer smart debugging suggestions when errors occur.

1.6 Expected Outcome & Benefits *(60–80 words)*

This platform presents a highly useful collaboration tool that promotes technical discipline and learning amongst developers. Through the incorporation of **Agora-based voice chat** and **ChatBox**, the platform facilitates easy communication between team members. The deliberate limitation of the **Grok 4.5 AI** to only the debugging process makes it possible for users to engage in independent thinking and “hand practice” when working on the program’s first version. While this structure simplifies the complicated process of debugging without making users rely heavily on artificial intelligence for writing codes, it creates an optimal environment for teams to resolve difficult issues collectively while improving their skills at solving them independently.

1.7 Original Contribution Statement *(20–30 words)*

The new idea is an **AI Activation Logic** that incorporates conditional **voice and text chats** in real time. The limitation of **Grok AI** mentorship only during troubleshooting enables the retention of user-written codes while focusing on independent logical design.